Grammar Addendum — slash form + namespace prefix + arrow form (operator mandate, ; arrow form)

2026-06-09

2026-07-02

EXTENDS the §11.4.140 action-prefix design with additional equivalences. Every registered action (e.g. BACKGROUND, REMINDER) MUST be invocable in ALL of these EQUIVALENT forms — same action, same expansion, same execution:

#	Form	Example	Notes
1	`ACTION_NAME :: <rest>`	`BACKGROUND :: Do X`	bare `::` form (no namespace)
2	`PREFIX::ACTION_NAME :: <rest>`	`DEFAULT::BACKGROUND :: Do X`	namespaced `::` form
3	`/ACTION_NAME <rest>`	`/BACKGROUND Do X`	bare slash form — ONLY if `/ACTION_NAME` does NOT collide with an existing/built-in slash command of the host agent
4	`/PREFIX::ACTION_NAME <rest>`	`/DEFAULT::BACKGROUND Do X`	namespaced slash form — disambiguates from any existing `/ACTION_NAME`; ALWAYS safe
5	`ACTION_NAME ---> <rest>` (and `PREFIX::ACTION_NAME ---> <rest>`)	`BACKGROUND ---> Do X` / `DEFAULT::BACKGROUND ---> Do X`	arrow form — the `--->` body separator (one ASCII space on each side); a third equivalent delimiter alongside `::` and `/`

Namespace prefix

- `PREFIX` is an action NAMESPACE. The reserved default namespace is **`DEFAULT`**. Custom namespaces are permitted (e.g. a project/plugin namespace) so the same action can run under the default OR a custom prefix.
- An action is executable WITH or WITHOUT the prefix: `BACKGROUND ::` ≡ `DEFAULT::BACKGROUND ::` ≡ `/BACKGROUND` ≡ `/DEFAULT::BACKGROUND` ≡ `BACKGROUND --->` ≡ `DEFAULT::BACKGROUND --->`.

- Namespace separator inside the token is `::` (no surrounding spaces): `PREFIX::ACTION_NAME`. This is DISTINCT from the action-body separator `::` (spaces both sides) used in forms 1/2 between the token and `<rest>`, and the arrow-body separator `--->` (spaces both sides) used in form 5.

Grammar (extended, anchored to first non-blank line)

- `::`-form regex: `^(?:([A-Z][A-Z0-9_]*)::)?([A-Z][A-Z0-9_]*) :: (.*)$` → group1=optional PREFIX, group2=ACTION_NAME, group3=rest.
- slash-form regex: `^/(?:([A-Z][A-Z0-9_]*)::)?([A-Z][A-Z0-9_]*)\s+(.*)$` → group1=optional PREFIX, group2=ACTION_NAME, group3=rest.
- arrow-form regex: `^(?:([A-Z][A-Z0-9_]*)::)?([A-Z][A-Z0-9_]*) ---> (.*)$` → group1=optional PREFIX, group2=ACTION_NAME, group3=rest. The `--->` separator is anchored immediately after the leading token; a mid-prose `--->` after a colon token (e.g. `A :: B ---> C`) parses as the COLON form (the arrow match falls through), so the `::`, arrow, and slash forms never cross-interfere.
- Token case: ACTION_NAME and PREFIX are UPPERCASE `[A-Z][A-Z0-9_]*` (directive/macro convention — DISTINCT from the §11.4.29 lowercase file/dir naming, which still governs the registry FILE names + script names). `DEFAULT` is the reserved default-namespace literal.
- Escape: leading `\` makes the line literal (no expansion): `\BACKGROUND :: x`, `\BACKGROUND ---> x`, and `\/BACKGROUND x`.
- Unknown grammar-shaped token (looks like an action but not in the registry) → ask per §11.4.66/§11.4.105, NEVER silently expand/drop (§11.4.6).

Conflict rule (slash form)

- `/ACTION_NAME` (form 3) is honored as the action ONLY when `ACTION_NAME` (case-folded for the collision check) does not match a built-in/host slash command. The registry MAY carry a per-action `slash_bare: true|false|auto` and a `slash_conflicts: [..]` list; `auto` = enable bare slash unless a known host command collides.
- Form 4 (`/PREFIX::ACTION_NAME`) is ALWAYS unambiguous and always honored.

Registry schema additions

```yaml
grammar:
  default_namespace: DEFAULT
  namespace_separator: '::'       # inside the token, no spaces
  body_separator: ' :: '          # token <-> rest, spaces both sides ('::' form)
  arrow_body_separator: ' ---> '  # token <-> rest, spaces both sides (arrow form)
  slash_prefix: '/'
  slash_body_separator: ' '       # token <-> rest (slash form)
  # regexes: colon_form_regex / slash_form_regex / arrow_form_regex
actions:
  - name: BACKGROUND
    namespaces: [DEFAULT]         # which namespaces this action is registered under
    slash_bare: auto              # bare /BACKGROUND honored unless host-command collision
    slash_conflicts: []           # known colliding host slash commands (forces /
                                  DEFAULT::BACKGROUND)
    # ... expansion, rules, composes_with as before
  - name: REMINDER
    namespaces: [DEFAULT]         # second registered action (verify-don't-assume status re-
                                  surfacing)
    slash_bare: auto
    slash_conflicts: []
    # ... expansion, rules, composes_with
```

Impl impact

- `apx_parse_prefix` MUST recognize all 5 forms, returning (namespace|DEFAULT, action, rest, matched_form ∈ {colon, slash, arrow}). The python and awk parse paths stay byte-identical; the awk arrow branch FALLS THROUGH to the colon check on an invalid leading token so `A :: B ---> C` parses identically in both paths.
- `apx_expand_prompt` resolves namespace+action against the registry; bare-slash honored per the conflict rule; the arrow form resolves to the SAME action/expansion as the `::` and slash forms.
- LAYER-2 hook handles all 5 first-line forms.
- The slash-command generator already emits `/background` (form 3) per agent; ADD the namespaced `/default::background` (form 4) where the agent's command syntax allows `::` (else a `default-background` alias) + document the collision fallback. The arrow form (5) is a free-form/typed delimiter, not a generated slash command.
- recognition_instruction.md teaches the agent all 5 forms + the conflict rule + escape + the `REMINDER` action.

Naming: action/namespace tokens UPPERCASE; FILES/DIRS lowercase snake_case (§11.4.29). DEFAULT is the reserved default namespace literal.